Cheat Sheet — Sesión 7: Personajes animados 3D

Desarrollo de videojuegos 2D y 3D · Godot 4 / GDScript

Del 2D al 3D

2D	3D
Node2D	Node3D
CharacterBody2D	CharacterBody3D
Sprite2D / AnimatedSprite2D	MeshInstance3D / un modelo con AnimationPlayer
Camera2D	Camera3D (con perspectiva)
Y crece **hacia abajo**	Y crece **hacia arriba**; el suelo es el plano XZ; adelante es **−Z**

Orden de armado (no se lo salten)

1. **WorldEnvironment** (*Environment* → *New Environment*, *Background* **Sky**) y **DirectionalLight3D** (*Shadow* → **Enabled**). Sin esto: pantalla negra.
2. **Suelo:** StaticBody3D → MeshInstance3D (*PlaneMesh* 30 × 30) + CollisionShape3D (*BoxShape3D* 30 × 0.2 × 30, Position Y −0.1).
3. **Camera3D** y escena principal (F5 → *Select Current*).
4. **Input Map:** *move_left*, *move_right*, *move_up*, *move_down* (flecha y letra). El Input Map del proyecto 2D no se hereda.
5. **Personaje:** CharacterBody3D → arrastrar el *.glb* como hijo → renombrarlo **Modelo** → CollisionShape3D con *CapsuleShape3D* (radio 0.35, altura 1.75, Position Y 0.875).

El personaje de Kenney

- *Personaje Kenney.zip* → descomprimir en el proyecto: *personaje/character-male-a.glb* y *personaje/Textures/colormap.png* (si falta la textura, sale blanco).

- Mide 0.67 m: Modelo → **Scale 2.6**. Se escala el modelo, **nunca** la forma de colisión.
- Mira hacia **+Z**: Modelo → **Rotation Y = 180** (si no, camina de espaldas).
- **Nombres reales:** doble clic en el *.glb* → *Advanced Import Settings* → **AnimationPlayer**: *idle*, *walk*, *sprint*, *jump*, *fall*, *die*, *crouch*… en minúscula.
- **Loop:** en esa ventana, clic en la animación → **Settings** → **Loop Mode: Linear** → **Reimport**. Sin loop se congela y patina.
- Ruta del AnimationPlayer: `$Modelo/AnimationPlayer` (está **dentro** del Modelo).

Moverse relativo a la cámara

```
var input_dir = Input.get_vector("move_left", "move_right", "move_up", "move_do
wn")
var entrada = Vector3(input_dir.x, 0.0, input_dir.y)   # la y del Vector2 va a
la z
# Solo el giro horizontal de la cámara (la inclinación no deforma la diagonal):
var direction = entrada.rotated(Vector3.UP, camara.global_rotation.y).normalize
d()
velocity.x = direction.x * speed
velocity.z = direction.z * speed
```

La versión de la demo (`camara.global_basis * entrada`, `direction.y = 0`, `normalized()`) funciona, pero con la cámara inclinada 35° la diagonal W+D sale (2.32, −1.9) en vez de (2.12, −2.12).

Correr

```
var speed = speed_correr if Input.is_action_pressed("correr") else speed_camina
r
```

`a if condición else b` es un `if/else` de una línea que devuelve un valor.

Girar suave

```
if direction.length() > 0:
    var angulo_objetivo = atan2(-direction.x, -direction.z)   # para un nodo qu
e mira a −Z
    rotation.y = lerp_angle(rotation.y, angulo_objetivo, giro * delta)
```

- `rotation.y` va en **radianes** (`deg_to_rad(90)` = 1.57).
- Con `giro` 10 a 60 cuadros por segundo, cada cuadro avanza 1/6 de lo que falta: −15°, −27.5°, −37.9°…
- `lerp_angle` va por el camino corto del círculo (de 170° a −170° son 20°); `lerp` no.
- Fuera del `if`: sin teclas `atan2(-0.0, -0.0)` da −180° y se voltea solo.

Saltar en 3D

```
if not is_on_floor():
    velocity += get_gravity() * delta      # get_gravity() = (0, -9.8, 0)
elif Input.is_action_just_pressed("saltar"):
    velocity.y = fuerza_salto              # POSITIVO en 3D
```

Altura ≈ $v^2 \div (2 \times 9.8)$. Con 5 m/s: 1.28 m en la cuenta, 1.318 m en el juego.

Estados y animación

```
enum Estado { QUIETO, CAMINAR, CORRER, AIRE }

func elegir_estado():          # después de move_and_slide()
    if not is_on_floor():
        return Estado.AIRE
    var plana = Vector2(velocity.x, velocity.z).length()
    if plana < 0.1:
        return Estado.QUIETO
    if plana > speed_caminar + 0.1:
        return Estado.CORRER
    return Estado.CAMINAR
```

La animación se elige mirando la **velocidad real** (después de `move_and_slide()`), no las teclas: contra una pared la velocidad es 0 y le toca *idle*.

La plantilla del juego: los enchufes

Variable de *personaje.gd*	Qué hace	Ejemplo desde otro script (pj = get_parent())
`multiplicador`	Multiplica la rapidez de las teclas	`pj.multiplicador = 0.5` (agachado), `0` (quieto)
`empuje`	Velocidad que se suma, en m/s	`pj.empuje = -pj.basis.z * 20` (dash hacia adelante)
`accion`	Animación que reemplaza a *idle* o *walk*	`pj.accion = "attack-melee-right"`; `""` = ninguna

`-pj.basis.z` es hacia donde mira el personaje. Lo que se cambia al empezar la acción se deshace al terminarla.

Arranque y frenado

```
var avance = Vector3.ZERO     # fuera de la función

var paso = speed / tiempo_arranque * delta
avance = avance.move_toward(direction * speed * multiplicador, paso)
velocity.x = avance.x + empuje.x
velocity.z = avance.z + empuje.z
```

Llega a la rapidez en `tiempo_arranque × 60` cuadros. En 2.5D es igual con un número: `avance = move_toward(avance, lado * speed, paso)`.

Una acción con tiempo (nodo hijo del Personaje)

```
enum Estado { LISTA, ACTIVA, RECARGA }

    tiempo -= delta
    match estado:
        Estado.LISTA:
            if Input.is_action_just_pressed(tecla) and puede():
                empezar()                # tiempo = duracion
        Estado.ACTIVA:
            if tiempo <= 0.001:
                terminar()               # tiempo = recarga
        Estado.RECARGA:
            if tiempo <= 0.001:
                estado = Estado.LISTA
```

Segundos × 60 = cuadros. Se compara con 0.001 porque restar 1/60 no da exacto 0. Para **mantener** una tecla: `is_action_pressed` en LISTA y terminar si la sueltan.

Otro personaje que se mueve solo

```
var hacia = objetivo.global_position - global_position
hacia.y = 0.0
if hacia.length() > 1.0:
    var dir = hacia.normalized()
    velocity.x = dir.x * rapidez
    velocity.z = dir.z * rapidez
    basis = Basis.looking_at(dir)
```

Patrullar y perseguir: PATRULLA → PERSEGUIR si `distancia < radio_ver`; PERSEGUIR → PATRULLA si `distancia > radio_perder`, y `radio_perder` mayor que `radio_ver` (histéresis). Para huir, la flecha al revés: `-hacia`. El Objetivo se elige en la **instancia** dentro de *main.tscn*.

2.5D: un mundo 3D visto de lado

- Camera3D con **Projection: Orthogonal** y **Size** 10 (los metros de alto que se ven), mirando hacia −Z.
- Personaje y enemigos con **Axis Lock → Linear Z**: nunca se salen del plano z = 0.
- Mover con `Input.get_axis("move_left", "move_right")` y mirar con `rotation.y = -PI / 2 * sign(lado)`.
- Salto con altura: `velocity.y = sqrt(2 * gravedad * altura_salto)`. Con g 20 y 1.5 m: 7.75 m/s, y el juego llega a 1.57 m.
- Para pixel art: un **Sprite3D** o **AnimatedSprite3D** con *Billboard* en vez del modelo.

Mixamo (opcional)

- mixamo.com (cuenta de Adobe) → personaje → *Idle* → **FBX**, **With Skin**. *Walking* → **Without Skin**.
- `anim.add_animation_library("walk", load("res://Walking.fbx"))` en `_ready()`, dentro de `if not anim.has_animation_library("walk"):`.
- Nombres: `mixamo_com` (idle) y `walk/mixamo_com`. Loop en *Advanced Import*. También mira hacia +Z: Rotation Y = 180.

El repositorio del equipo (desde el Taller 7)

- **Uno por equipo**, público, creado desde la plantilla por una sola persona; los demás *Clone*. La URL va una vez en la hoja *Repositorio del juego* del Excel.
- **Su cuenta** en GitHub Desktop (*File → Options → Accounts*), en **su** PC: el autor del commit es quien se califica.
- **Cada tarea en su archivo**: A edita *personaje.gd*; B, el script de su nodo hijo; C, la escena y el script de su

enemigo. En el Taller 8: la cámara en *camara_sigue.gd* o *camara_primera.gd*; *interactuar.gd* y *apuntar.gd* en nodos hijos. Nadie edita el archivo de otro.

- **Un commit por nivel**, con el mensaje exacto de la guía: `T7 N2 - Tarea A - arranque y frenado`. *Fetch* y *Pull* antes de empezar; **Push origin** después de cada commit.
- **El documento** lleva usuario de GitHub, URL del repo y el enlace a cada commit; el código no se copia.

Errores típicos

Síntoma	Causa
Pantalla negra	Falta el Environment del WorldEnvironment, o falta la Camera3D
El personaje cae sin fin	El suelo no tiene CollisionShape3D, o la forma no está dentro del Suelo
"The InputMap action ... doesn't exist"	La acción no se creó o se escribió distinto
"... 'play' on a base object of type 'Nil'"	`$Modelo/AnimationPlayer` no calza: el Modelo se llama distinto
Camina de espaldas	Falta la Rotation Y = 180 del Modelo, o `atan2` sin los signos menos
La animación se congela	Falta el loop en *Advanced Import*
Personaje blanco	La textura no está en *personaje/Textures/*
"Animation not found"	El nombre de la animación está en mayúscula o mal escrito
No salta	`velocity.y` del salto negativa, o la acción *saltar* no existe
Salta en el aire	Falta preguntar `is_on_floor()` antes del salto
La rapidez se queda en 0.17	`var avance` está dentro de `_physics_process()`
La acción se repite al mantener la tecla	`is_action_pressed` en vez de `is_action_just_pressed`
El enemigo no se mueve ("... type 'Nil'")	Falta elegir el Objetivo en la instancia de *main.tscn*
En 2.5D se sale del plano	Falta Axis Lock Linear Z
El commit sale a nombre de otro	GitHub Desktop está con la cuenta de otro: *Options* → *Accounts*
GitHub Desktop avisa un conflicto	Dos personas tocaron el mismo archivo; avisen al profesor